

SESIÓN

13

Implementando aplicaciones N capas con EF y ASP NET Framework

UNIDAD DE APRENDIZAJE 4

- Implementando el CRUD de tablas base con el control GridView y ModalPopup de AJAX Toolkit
- Implementando consultas con valor de negocio con ASP NET Framework
- Paginación en los GridView
- Paginación con Procedimientos almacenados

/ INTRODUCCIÓN

- En las aplicaciones WEB también podemos hacer CRUD y consultas con valor de negocio, y lo más importante, listados con paginación. Dada la naturaleza de este tipo de aplicaciones, se hace necesario generar listados donde la presentación de los registros se pague y esto se pueden hacer de 2 maneras: mediante el control GridView y por medio de procedimientos almacenados, siendo esta última la forma más recomendable.
- Al culminar la presente podrás:
 - Implementar el mantenimiento de una tabla desde formularios ASP. Net reutilizando las capas funcionales ADO, BL y el proyecto de entidades de negocio BE.
 - Interacción con ModalPopup AJAX para agilizar las ventanas de inserción y actualización de registros.
 - Implementar consultas de valor en aplicaciones WEB y consultas que muestren la información en gráficos estadísticos
 - Implementar la paginación local y la paginación basada en procedimientos almacenados para mejor presentación de los registros en los listados.

¡¡¡A divertirnos !!!

**/ IMPLEMENTANDO EL CRUD DE TABLAS BASE
CON EL CONTROL GRIDVIEW Y MODALPOPUP DE
AJAX TOOLKIT**

/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

Definir el diseño de la plantilla para el CRUD

- Como parte de la implementación del mantenimiento de la tabla (CRUD) basándonos en lo hecho en la capa de presentación Windows, ahora en la capa de presentación WEB, se deberá adaptar una plantilla de formulario (que puede ser proporcionada por su instructor o elaborarla tú mismo) de tal manera que se tenga un diseño final similar a los siguientes, que están basados en un CRUD para una tabla de proveedores:

MANTENIMIENTO DE PROVEEDORES							
Nuevo proveedor							
Ingrese razon social:							
Codigo	Razon social	Direccion	Telefono	Departamento	Provincia	Distrito	Estado
V003	3M	Av. Venezuela 550	2908165	LIMA	LIMA	LOS OLIVOS	Inactivo
V137	ACABADOS IR	CALLE JULIOCESAR 723		AREQUIPA	CAYLLOMA	LLUTA	Activo
V005	Acker	Calle Las Amapolas 600	4780143	LIMA	CALLAO	VENTANILLA	Activo
V099	ADONAI	AV MARIA AUXILIADORA 568	2658503	LIMA	LIMA	COMAS	Inactivo
V172	ALBERTO OTINIANO PASTOR	Av. El campo Verde 239		AREQUIPA	CAYLLOMA	ACHOMA	Activo
V192	ALBRESA AGUAS SAC			AREQUIPA	AREQUIPA	ALTO SELVA ALEGRE	Activo
V163	AMERICAN SERVICE 4 HOME SRL	JR.ANTONIO CABO 867	5332069	AREQUIPA	CAYLLOMA	MACA	Activo
V189	ANDRES QUISPE PANIAGUA	Av. ISIL 1730	27	AREQUIPA	CAMANA	MARISCAL CACERES	Activo
V191	ANIBAL DOLORES ALVAREZ			AREQUIPA	CAYLLOMA	MACA	Activo
V023	AROCA	Urb. LOS ROSALES Mzna B Lote 11	-	LIMA	LIMA	BREÑA	Activo
V114	ARONES HUALLAGA NILO	CALLE LOS PANCHECHOS 177		LIMA	LIMA	BARRANCO	Activo
V050	ASPEN INVERSIONES S.A	JR. PEDRO RUIZ GALLO 2063	3561072	LIMA	LIMA	CIENEGUILLA	Activo
V002	Atlas	Av. Lima 471	9874560	LIMA	LIMA	JESUS MARIA	Activo
V024	AZEMAR PERU SRL	SAN ENRIQUE 980	4587581	LIMA	LIMA	SANTIAGO DE SURCO	Activo

/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

Definir el diseño de la plantilla para el CRUD

- Con el botón “Nuevo proveedor” se mostrará un control AJAX de tipo ModalPopup para la inserción de nuevos registros. Tal como se detalla a continuación :

MANTENIMIENTO DE PROVEEDORES

Nuevo proveedor

Ingrese razon social:

Codigo	Razon social	Estado
V003	3M	Inactivo
V137	ACABADOS IR	Activo
V005	Acker	Activo
V099	ADONAI	Inactivo
V172	ALBERTO OTINIANO PASTOR	Activo
V192	ALBRESA AGUAS SAC	Activo
V163	AMERICAN SERVICE 4 HOME SP	Activo
V189	ANDRES QUISPE PANIAGUA	Activo
V191	ANIBAL DOLORES ALVAREZ	Activo
V023	AROCA	Activo
V114	ARONES HUALLAGA NILO	Activo
V050	ASPEN INVERSIONES S.A	Activo

Nuevo proveedor

Razon social:

Direccion:

Telefono:

RUC:

Departamento:

Provincia:

Distrito:

Representante de ventas:

Estado: ☒ Activo

Grabar **Cancelar**

/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

Definir el diseño de la plantilla para el CRUD

- Para el caso de la actualización se procederá hacer click en la columna con la imagen del lápiz (puede ser cualquier otra imagen si desea) dentro del GridView, para con ello mostrar otro control ModalPopup para la respectiva edición del registro, como se muestra a continuación:

MANTENIMIENTO DE PROVEEDORES

Nuevo proveedor

Ingrese razon social:

	Codigo	Razon social
	V003	3M
	V137	ACABADOS IR
	V005	Acker
	V099	ADONAI
	V172	ALBERTO OTINIANO PASTOR
	V192	ALBRESA AGUAS SAC
	V163	AMERICAN SERVICE 4 HOME SRL
	V189	ANDRES QUISPE PANIAGUA
	V191	ANIBAL DOLORES ALVAREZ

MANTENIMIENTO DE PROVEEDORES

Nuevo proveedor

Ingrese razon social:

	Codigo	Razon social	Estado
	V003	3M	Inactivo
	V137	ACABADOS IR	Activo
	V005	Acker	Activo
	V099	ADONAI	Inactivo
	V172	ALBERTO OTINIANO PASTOR	Activo
	V192	ALBRESA AGUAS SAC	Activo
	V163	AMERICAN SERVICE 4 HOME SP	Activo
	V189	ANDRES QUISPE PANIAGUA	Activo
	V191	ANIBAL DOLORES ALVAREZ	Activo
	V023	AROCA	Activo
	V114	ARONES HUALLAGA NILO	Activo

Actualizar proveedor

Codigo: **V003**

Razon social:

Direccion:

Telefono:

RUC:

Departamento:

Provincia:

Distrito:

Representante de ventas:

Estado: ☐ Activo

CALLE LOS PANCHECHOS 177 LIMA LIMA BARRANCO

/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

- En el evento LOAD del formulario se cargará el GridView con los proveedores manejando la posibilidad del filtrado:

```
public partial class WebMantProveedores : System.Web.UI.Page
{
    // Instancias...
    ProveedorBE objProveedorBE = new ProveedorBE();
    ProveedorBL objProveedorBL = new ProveedorBL();
    UbigeoBL objUbigeoBL = new UbigeoBL();
    DataView dtv;

    protected void Page_Load(object sender, EventArgs e)
    {
        try
        {
            if (Page.IsPostBack == false)
            {
                CargarDatos("");
            }
        }
        catch (Exception ex)
        {
            lblMensajePopup.Text = "Error : " + ex.Message;
            PopMensaje.Show();
        }
    }

    private void CargarDatos(String strFiltro)
    {
        // Creamos la vista en memoria y cargamos los datos
        dtv = new DataView(objProveedorBL.ListarProveedor());
        dtv.RowFilter = "raz_soc_prv like '%" + strFiltro + "%'";
        grvProveedores.DataSource = dtv;
        grvProveedores.DataBind();
        if (grvProveedores.Rows.Count == 0)
        {
            lblMensajePopup.Text = "No existen registros con el criterio ingresado.";
            PopMensaje.Show();
        }
    }
}
```

```
protected void btnFiltrar_Click(object sender, ImageClickEventArgs e)
{
    try
    {
        CargarDatos(txtFiltro.Text);
    }
    catch (Exception ex)
    {
        lblMensajePopup.Text = ex.Message;
        PopMensaje.Show();
    }
}
```

/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

- Para insertar un nuevo proveedor debemos preparar el modal popup correspondiente, limpiando los controles y cargando el ubigeo.

```
protected void btnNuevo_Click(object sender, EventArgs e)
{
    try
    {
        // Limpiamos todos los controles del panel1 asociados al modalpopup
        // de insercion (PopMan01)
        lblMensaje1.Text = "";
        txtRS1.Text = "";
        txtDir1.Text = "";
        txtRUC1.Text = "";
        txtTel1.Text = "";
        txtRepVen1.Text = "";
        // Activo por defecto al insertar
        chkEstado1.Checked = true;
        // Cargamos el ubigeo
        CargarUbigeo(1, "14", "01", "01"); // Lima, Lima , Lima por defecto
        // Mostramos el PopMant
        txtRS1.Focus();
        PopMan01.Show();
    }
    catch (Exception ex)
    {
        lblMensajePopup.Text = ex.Message;
        PopMensaje.Show();
    }
}
```

```
private void CargarUbigeo(int16 intAccion, String IdDepa, String IdProv, String IdDist)
{
    // Combos de ubigeo para el CRUD del proveedor
    if (intAccion == 1) // Combos de insercion
    {
        cboDepartamento1.DataSource = objUbigeoBL.Ubigeo_Departamentos();
        cboDepartamento1.DataValueField = "IdDepa";
        cboDepartamento1.DataTextField = "Departamento";
        cboDepartamento1.DataBind();
        cboDepartamento1.SelectedValue = IdDepa;
        cboProvincia1.DataSource = objUbigeoBL.Ubigeo_ProvinciasDepartamento(IdDepa);
        cboProvincia1.DataValueField = "IdProv";
        cboProvincia1.DataTextField = "Provincia";
        cboProvincia1.DataBind();
        cboProvincia1.SelectedValue = IdProv;
        cboDistrito1.DataSource = objUbigeoBL.Ubigeo_DistritosProvinciaDepartamento(IdDepa, IdProv);
        cboDistrito1.DataValueField = "IdDist";
        cboDistrito1.DataTextField = "Distrito";
        cboDistrito1.DataBind();
        cboDistrito1.SelectedValue = IdDist;
    }
    else if (intAccion == 2) // Combos de actualizacion
    {
        cboDepartamento2.DataSource = objUbigeoBL.Ubigeo_Departamentos();
        cboDepartamento2.DataValueField = "IdDepa";
        cboDepartamento2.DataTextField = "Departamento";
        cboDepartamento2.DataBind();
        cboDepartamento2.SelectedValue = IdDepa;
        cboProvincia2.DataSource = objUbigeoBL.Ubigeo_ProvinciasDepartamento(IdDepa);
        cboProvincia2.DataValueField = "IdProv";
        cboProvincia2.DataTextField = "Provincia";
        cboProvincia2.DataBind();
        cboProvincia2.SelectedValue = IdProv;
        cboDistrito2.DataSource = objUbigeoBL.Ubigeo_DistritosProvinciaDepartamento(IdDepa, IdProv);
        cboDistrito2.DataValueField = "IdDist";
        cboDistrito2.DataTextField = "Distrito";
        cboDistrito2.DataBind();
        cboDistrito2.SelectedValue = IdDist;
    }
}
```


/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

- No olvide programar el evento `SelectedIndexChanged` de los combos de departamento y provincia del `ModalPopup` de inserción para el efecto cascada del ubigeo....

```
protected void cboDepartamento1_SelectedIndexChanged(object sender, EventArgs e)
{
    CargarUbigeo(1,cboDepartamento1.SelectedValue.ToString(), "01", "01");
    //Para que al postear el evento SelectedIndexChanged se mantenga abierto el PopMan01
    PopMan01.Show();
}

protected void cboProvincia1_SelectedIndexChanged(object sender, EventArgs e)
{
    CargarUbigeo(1,cboDepartamento1.SelectedValue.ToString(),
                cboProvincia1.SelectedValue.ToString(), "01");
    //Para que al postear el evento SelectedIndexChanged se mantenga abierto el PopMant
    PopMan01.Show();
}
```

Nota: Será importante asignar el valor true a la propiedad `AutoPostBack` de cada combo

/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

- Para grabar el nuevo proveedor debemos configurar la entidad de negocio y llamar al método InsertarProveedor, tal como lo hacíamos en la capa de presentación en aplicaciones Windows:

```
protected void btnGrabar1_Click(object sender, EventArgs e)
{
    try
    {
        // Puede hacer las validaciones por código (o agregar RequiredFieldValidator)
        // Como ejemplo solo validaremos la Razon Social
        if (txtRS1.Text.Trim() == "")
        {
            throw new Exception("La razon social es obligatoria.");
        }

        // Tras las validaciones se configura la instancia de objProveedorBE y se invoca al
        // metodo Insertar del objProveedorBL
        objProveedorBE.Raz_soc_prv = txtRS1.Text;
        objProveedorBE.Dir_prv = txtDir1.Text;
        objProveedorBE.Ruc_prv = txtRUC1.Text;
        objProveedorBE.Tel_prv = txtTel1.Text;
        objProveedorBE.Rep_ven = txtRepVen1.Text;
        objProveedorBE.Id_Ubigeo = cboDepartamento1.SelectedValue +
            cboProvincia1.SelectedValue + cboDistrito1.SelectedValue;
        objProveedorBE.Usu_Registro = "Jleon";
        objProveedorBE.Est_prv = Convert.ToInt16(chkEstado1.Checked);
        // Insertamos el registro...
        if (objProveedorBL.InsertarProveedor(objProveedorBE) == true)
        {
            // Refrescamos el listado
            CargarDatos(txtFiltro .Text );
        }
        else
        {
            throw new Exception("No se inserto el registro. Contacte con IT.");
        }
    }
    catch (Exception ex)
    {
        lblMensaje1.Text = ex.Message;
        PopMan01.Show(); // para mantener el popup abierto tras el clic en Grabar1
    }
}
```

/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

- Para actualizar un proveedor debemos seleccionarlo desde el GridView, haciendo clic en el ColumnButton “Editar” del GridView (la columna del lápiz) y mostrando en el Modal Popup respectivo :

```
protected void grvProveedores_RowCommand(object sender, GridViewCommandEventArgs e)
{
    try
    {
        // Se obtiene en que fila del grid se ha seleccionado el boton
        Int16 fila = Convert.ToInt16(e.CommandArgument);
        // Validamos si el boton tiene el nombre Editar
        if (e.CommandName == "Editar")
        {
            // Limpiamos el lblMensaje2
            lblMensaje2.Text = "";
            // Obtenemos el codigo del proveedor de la celda 1 de la fila seleccionada
            String strCod = grvProveedores.Rows[fila].Cells[1].Text;
            // Obtenemos la instancia del proveedor a actualizar
            objProveedorBE = objProveedorBL.ConsultarProveedor(strCod);
            // Mostramos el proveedor en los controles del panel del PopMan02 (actualizacion del registro)
            lblCod.Text = objProveedorBE.Cod_prv;
            txtRS2.Text = objProveedorBE.Raz_soc_prv;
            txtDir2.Text = objProveedorBE.Dir_prv;
            txtRUC2.Text = objProveedorBE.Ruc_prv;
            txtTel2.Text = objProveedorBE.Tel_prv;
            txtRepVen2.Text = objProveedorBE.Rep_ven;
            chkEstado2.Checked = Convert.ToBoolean(objProveedorBE.Est_prv);
            String Id_Ubigeo = objProveedorBE.Id_Ubigeo;
            // Manejamos el ubigeo para el panel de PopMan02
            CargarUbigeo(2, Id_Ubigeo.Substring(0, 2), Id_Ubigeo.Substring(2, 4), Id_Ubigeo.Substring(4, 2));

            // Mostramos el PopMan02
            PopMan02.Show();
        }
    }
    catch (Exception ex)
    {
        lblMensajePopup.Text = ex.Message;
        PopMensaje.Show();
    }
}
```

/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

- No olvide programar el evento `SelectedIndexChanged` de los combos de departamento y provincia contenidos en este `ModalPopup` para el efecto cascada del ubigeo....

```
protected void cboDepartamento2_SelectedIndexChanged(object sender, EventArgs e)
{
    CargarUbigeo(2,cboDepartamento2.SelectedValue.ToString(), "01", "01");
    //Para que al postear el evento SelectedIndexChanged se mantenga abierto el PopMan02
    PopMan02.Show();
}

protected void cboProvincia2_SelectedIndexChanged(object sender, EventArgs e)
{
    CargarUbigeo(2,cboDepartamento2.SelectedValue.ToString(),
                cboProvincia2.SelectedValue.ToString(), "01");
    //Para que al postear el evento SelectedIndexChanged se mantenga abierto el PopMan02
    PopMan02.Show();
}
```

Nota : Aquí también recuerde asignar el valor true a la propiedad `AutoPostBack` de cada combo

/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

- Si queremos actualizar los datos del proveedor debemos codificar el botón “Grabar” del ModalPopup de actualización:

```
protected void btnGrabar2_Click(object sender, EventArgs e)
{
    try
    {
        // Validamos la razon social
        if (txtRS2.Text == "")
        {
            throw new Exception("La razon social es obligatoria.");
        }
        // Tras las validaciones se configura la instancia de objProveedorBE y se invoca al metodo Actualizar
        // del objProveedorBL
        objProveedorBE.Cod_prv = lblCod.Text;
        objProveedorBE.Raz_soc_prv = txtRS2.Text;
        objProveedorBE.Dir_prv = txtDir2.Text;
        objProveedorBE.Ruc_prv = txtRUC2.Text;
        objProveedorBE.Tel_prv = txtTel2.Text;
        objProveedorBE.Rep_ven = txtRepVen2.Text;
        objProveedorBE.Id_Ubigeo = cboDepartamento2.SelectedValue + cboProvincia2.SelectedValue + cboDistrito2.SelectedValue;
        objProveedorBE.Usu_Ult_Mod = "cmontoya";
        objProveedorBE.Est_prv = Convert.ToInt16(chkEstado2.Checked);
        // Actualizamos...
        if (objProveedorBL.ActualizarProveedor(objProveedorBE) == true)
        {
            CargarDatos(txtFiltro.Text);
        }
        else
        {
            throw new Exception("No se actualizo el registro. Contacte con IT.");
        }
    }
    catch (Exception ex)
    {
        lblMensaje2.Text = ex.Message;
        PopMan02.Show();
    }
}
```

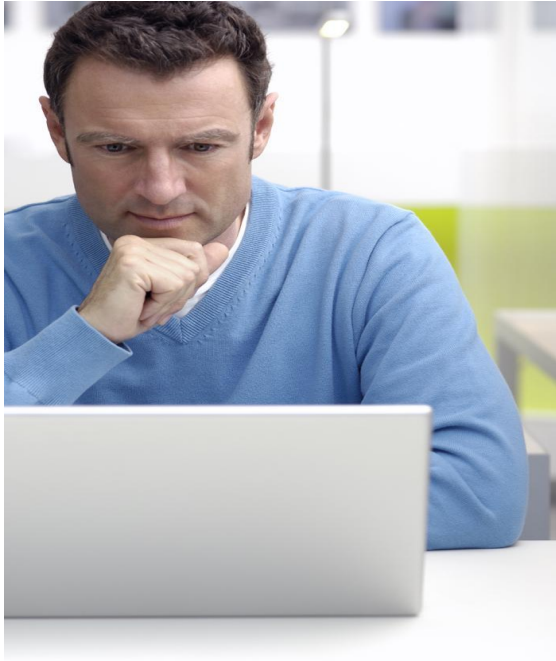
/ IMPLEMENTANDO EL CRUD DE TABLAS BASE CON EL CONTROL GRIDVIEW Y MODALPOPUP DE AJAX TOOLKIT

- Por último programe el evento RowDataBound del GridView para mostrar de color rojo los proveedores inactivos y dejar en color negro los activos.

```
protected void grvProveedores_RowDataBound(object sender, GridViewRowEventArgs e)
{
    try
    {
        // El evento recorre cada fila del grid cada vez que este se enlaza a su origen de datos
        if (e.Row.RowType == DataControlRowType.DataRow)
        {
            // Obtenemos el valor del campo Estado de cada fila (celda 8 )
            String estado = e.Row.Cells[8].Text;
            // Si esta inactivo el color de la fuente (ForeColor) de la fila estara en rojo
            if (estado == "Inactivo")
            {
                e.Row.ForeColor = System.Drawing.Color.Red;
            }
        }
    }
    catch (Exception ex)
    {
        lblMensajePopup.Text = ex.Message;
        PopMensaje.Show();
    }
}
```

/ LABORATORIO

Laboratorio 1: Implementar un CRUD con capa de presentación WEB



En este laboratorio aprenderemos a implementar un CRUD en una aplicación WEB ASP Net Framework empleando controles de AJAX Toolkit.

**/ IMPLEMENTANDO CONSULTAS CON VALOR DE
NEGOCIO CON ASP NET FRAMEWORK**

Creando consultas del sistema

- Uno de los *elementos más valiosos de un sistema de información* es precisamente eso, **la información**.
- Como ya se revisó en la sesión 6, para que los usuarios puedan tomar decisiones efectivas, el sistema debe ser capaz de responder a inquietudes propias del día a día del negocio, para que en base al sustento de la calidad de información presentada, el usuario pueda ejecutar sus procesos de manera correcta.
- Será un elemento importante que el sistema permita obtener información **concisa, validada y sobre todo oportuna**.

/ IMPLEMENTANDO CONSULTAS CON VALOR DE NEGOCIO CON ASP NET FRAMEWORK

Creando consultas del sistema

- Podemos tomar como ejemplo la consulta de la facturación de un cliente entre 2 fechas, pero ahora en entorno WEB:

CONSULTA DE FACTURACION POR CLIENTE ENTRE FECHAS

Ingresa cliente: 

Razon Social: RUC:

Direccion: Ubigeo:

Telefono: Estado:

Deuda:

INGRESE FECHAS DE CONSULTA (* Obligatorios):

Fecha inicio (*): Fecha fin (*):

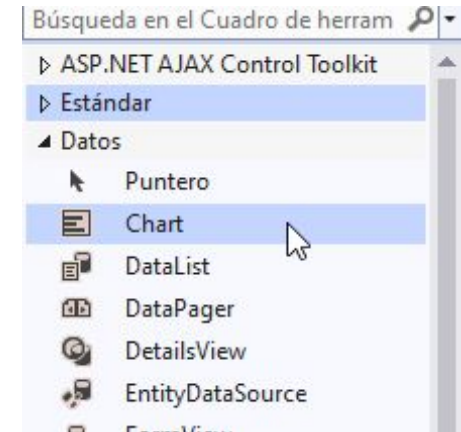
Cantidad de facturas : 5

[Retornar](#)

Nro. factura	Fec. facturacion	Fec. cancelacion	Total (S/.)	Estado	Apell. Vendedor	Nom. Vendedor
FA1223	9/10/2020		440.00	Pendiente	Zevallos	Vilma
FA1604	20/01/2021	15/02/2021	16,784.24	Cancelada	Sanchez Espinoza	Alexis
FA1711	17/12/2020		2,263.92	Pendiente	Soto Campos	Javier
FA1752	26/09/2020		2,887.14	Anulada	Abad Lopez	Eva Maria
FA1827	11/04/2021		3,560.00	Anulada	Herrera Herrera	Maria

Creando consultas con gráficos estadísticos

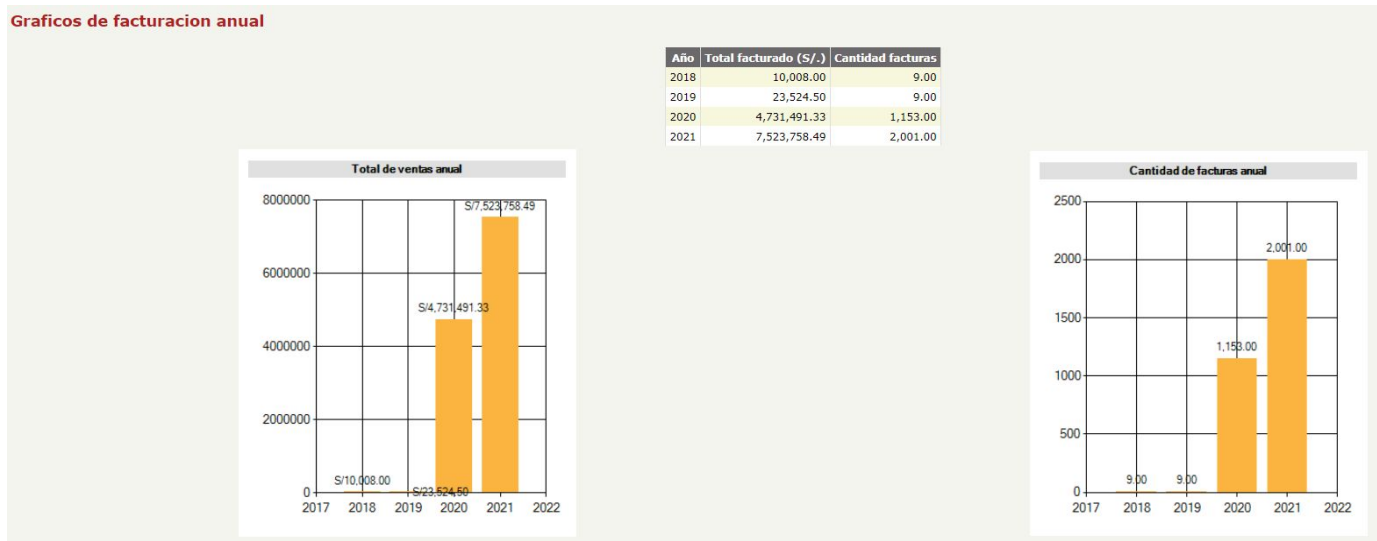
- No siempre la información que generen las consultas debe ser tabulares. Es importante también generar información representada en gráficos estadísticos, que denoten el comportamiento de las actividades más importantes del negocio, partiendo del refrán que dice “un gráfico vale más que mil palabras”.
- Para mostrar información en gráficos estadísticos **emplearemos el control Chart**, ubicado dentro del grupo Datos del cuadro de herramientas, tal como se muestra en el recuadro de la derecha.



/ IMPLEMENTANDO CONSULTAS CON VALOR DE NEGOCIO CON ASP NET FRAMEWORK

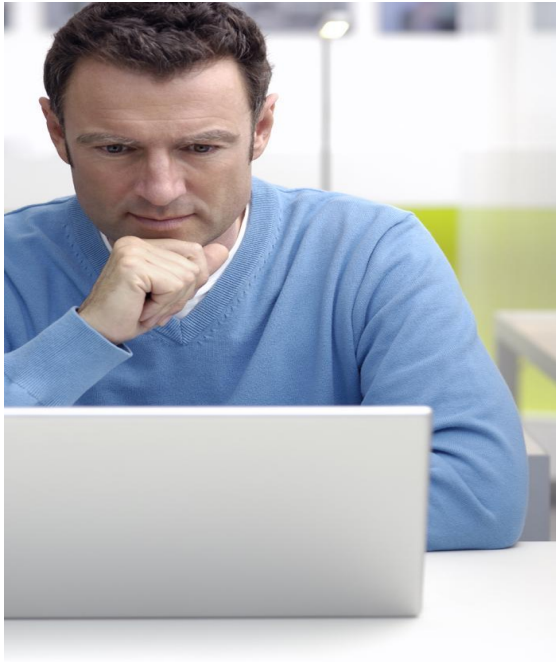
Creando consultas con gráficos estadísticos

- Por ejemplo, se pueden generar gráficos estadísticos con los totales facturados, así como la cantidad de facturas emitidas en cada año por la información registrada en el sistema, tal como se aprecia en el siguiente recuadro:



/ LABORATORIO

Laboratorio 2: Implementar consulta de valor de negocio con capa de presentación WEB



En este laboratorio aprenderemos a implementar una consulta de valor de negocio dentro de una aplicación WEB ASP Net Framework empleando controles de AJAX Toolkit.

/ PAGINACIÓN EN LOS GRIDVIEW

Paginando en un GridView

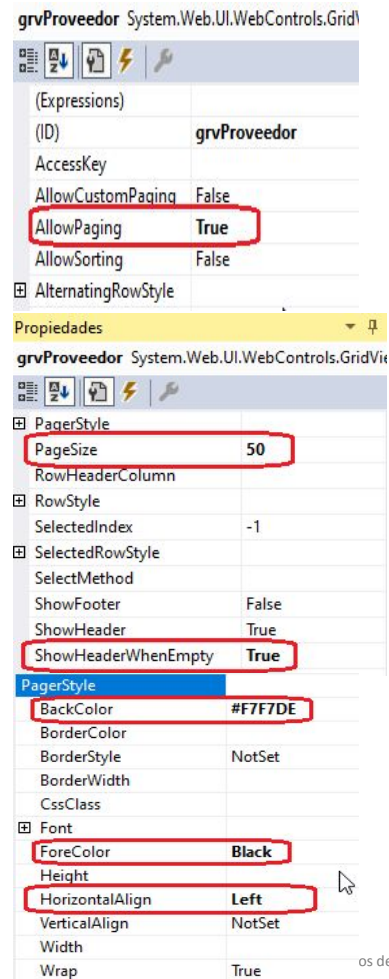
- Cuando hay muchos registros que mostrar en un GridView lo más recomendable es paginarlos, para que se muestren dichos registros en páginas de 20 en 20 (recomendable) o de 50 en 50.
- Hay 2 formas de hacer esto: paginando el GridView o paginando la consulta desde un procedimientos almacenados.
- En el mantenimiento de Proveedores emplearemos la primera manera, que si bien es cierto no es la más eficiente, para volúmenes de datos menores a 1000 es aceptable.
- Veremos en el próximo subtema la paginación desde procedimientos almacenados.

V050	ASPEN INVERSIONES S.A	JR. PEDRO RUIZ GALLO 2063	3561072	LIMA	LIMA	CIENEGUILLA	Activo
V002	Atlas	Av. Lima 471	9874560	LIMA	LIMA	JESUS MARIA	Activo
V024	AZEMAR PERU SRL	SAN ENRIQUE 980	4587581	LIMA	LIMA	SANTIAGO DE SURCO	Activo
V025	BACKUS & JOHNSTON UNION DE CERVECERIAS	AV. NICOLAS AYLLON 3986	0800199	LIMA	LIMA	PUNTA NEGRA	Activo
V020	Bari	Av. Arnaldo Marquez 1219	99876452	LIMA	LIMA	LINCE	Activo
V142	BAZAR ANTHONY	AV. GAMARRA 298		AREQUIPA	CAYLLOMA	LARI	Activo
V108	BLAVES INTERNACIONAL SAC	C.LAS ESMERALDAS 482 - BALCONCILLO	4714639	LIMA	LIMA	VILLA EL SALVADOR	Activo
V174	BODEGA ALEXIS PETRONILA BINOSCO CARTER	JR. LA VICTORIA 344		AREQUIPA	AREQUIPA	UCHUMAYO	Activo
V026	BOLSAS PERUANAS SA	CASA BLANCA 111	-	LIMA	LIMA	SANTIAGO DE SURCO	Activo

1 2 3 4 5 6 7 8 9 10 ...

Propiedades del GridView para permitir la paginación

- Si desea que el GridView permita paginación debe asignar el valor True a la propiedad AllowPage
- La definición del tamaño de página lo define en la propiedad PageSize y si desea que la cabecera del GridView esté siempre visible así no hayan registros que mostrar lo hace colocando en True la propiedad ShowHeaderWhenEmpty.
- Por último el estilo de la paginación lo define en el grupo de propiedades PagerStyle



El evento PageIndexChanging

- Para pasar de página en página en los registros del GridView debe programar el evento PageIndexChanging.
- A continuación mostramos nuevamente el código de dicho evento para nuestro ejemplo del mantenimiento de proveedores. Como se aprecia en la imagen se invoca al método CargarDatos para refrescar el GridView con la siguiente página:

```
protected void grvProveedores_PageIndexChanging(object sender, GridViewPageEventArgs e)
{
    grvProveedores.PageIndex = e.NewPageIndex;
    CargarDatos(txtFiltro.Text);
}
```

/ PAGINACIÓN CON STORE PROCEDURE

Entendiendo el escenario

- Se comprobó en el subtema anterior que los registros que se muestran en un control GridView pueden ser mostrados en lotes o “páginas” cuyo tamaño puede ser variable, recomendablemente entre 20 y 50 registros. Esto se aplica cuando tenemos una gran cantidad de registros que mostrar y no resulta amigable desplazarnos muy hacia la parte inferior para poder visualizar todos los registros.
- Si bien es cierto podemos aplicar filtros para reducir la cantidad de registros a mostrar, es posible siempre que la masa de registros necesite de la paginación.



Paginación con Store Procedure

- Pero en realidad, la paginación basada en el control GridView no es óptima en performance si la cantidad de registros devueltos por las consultas (filtradas o no) son por arriba de los mil registros.
- Si la consulta por ejemplo arroja 1500 registros, lo que hace en realidad el control GridView es que los 1500 se muestren de 50 en 50 (o del tamaño que se defina la página). Es como hacer una convocatoria a una reunión a 1500 personas y asistan todas a la sala de recepción y luego las agrupemos de 50 en 50. Como se darán cuenta la sala de recepción estará repleta y se generarían tumultos y problemas. Lo ideal sería que la convocatoria se haga a un primer grupo de 50 personas de tal manera que la sala de recepciones no se llene, y si necesitamos 50 más las invocamos haciendo una nueva convocatoria. Esto último es más eficiente y se puede lograr mediante consultas paginadas con Store Procedure.

Paginación con Store Procedure

- Para ello debemos manejar conceptos de filtros en la consulta (mediante uno o más parámetros) , definir el tamaño de página y en base a ello cuántas páginas se van a generar en base a la cantidad de registros resultantes.
- Los elementos más importantes en este tipo de escenarios son:
 - a) La elaboración del Store Procedure que maneje la paginación
 - b) La creación de los métodos en la capa de datos y de negocios que invoquen a la funcionalidad del Store Procedure.
 - c) Implementar la capa de presentación (formulario) que optimice la funcionalidad de la paginación y permita definir los parámetros y la navegación por las diferentes páginas generadas.

/ PAGINACIÓN CON STORE PROCEDURE

Los Store Procedure de paginación

```
CREATE OR ALTER PROCEDURE [dbo].[usp_NumPag_ListarFacturas_Paginacion]
    @Cod_cli varchar(4),
    @Cod_ven varchar(3),
    @Estado varchar(10),
    @NUMREG INT OUTPUT -- Parametro de salida
AS
    -- OBTIENE EL NUMERO DE REGISTROS DEVUELTOS DEL usp_ListarFacturas_Paginacion
    IF @Cod_cli='' AND @Cod_ven='' AND @Estado=''
        -- Si no hay valores significativos en los parametros devuelve 50 o si la
        -- cantidad de registros que existan
        BEGIN
            DECLARE @CONTADOR INT
            SET @CONTADOR=(SELECT COUNT (*) from vw_VistaFacturas)
            IF @CONTADOR>=50
                SET @NUMREG=50
            ELSE
                SET @NUMREG=@CONTADOR
            END
        ELSE
```

```
BEGIN
    -- En caso contrario se analizan los parametros pasados y se obtiene
    -- el numero de registros que cumplen las condiciones de filtrado
    IF @Cod_cli=''
        SET @Cod_cli='%'
    IF @Cod_ven=''
        SET @Cod_ven='%'
    IF @Estado=''
        SET @Estado='%'
    SET @NUMREG=
    ( SELECT COUNT (*) from vw_VistaFacturas
      where
        Cod_cli like '%' + @Cod_cli + '%'
        and Cod_ven like '%' + @Cod_ven + '%'
        and Estado like '%' + @Estado + '%')
END
```


/ PAGINACIÓN CON STORE PROCEDURE

Los Store Procedure de paginación

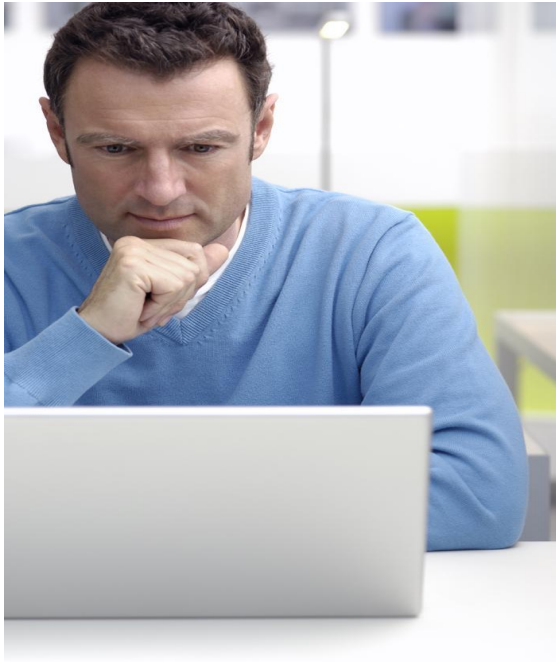
```
CREATE OR ALTER PROCEDURE [dbo].[usp_ListarFacturas_Paginacion]
@Cod_cli varchar(4),
@Cod_ven varchar(3),
@Estado varchar(10),
@NumPag int
AS
DECLARE @NUMREG INT
DECLARE @PAGESIZE INT
SET @PAGESIZE=50 --Tamaño de pagina

IF @Cod_cli='' AND @Cod_ven='' AND @Estado=''
BEGIN
    -- Si los valores de los parametros son cadenas en blanco ,
    -- se obtienen los primeros 50 registros
    SELECT top 50 Num_fac,Fec_fac,Fec_can,Cod_cli,Raz_soc_cli,Ruc_cli,
    Departamento,Provincia ,Distrito,
    Cod_ven, Ape_ven ,Nom_ven ,Total, Estado
    from vw_VistaFacturas
    ORDER BY Num_fac DESC
END
ELSE
```

```
BEGIN
    -- En caso contrario se analizan los parametros pasados
    IF @Cod_cli=''
        SET @Cod_cli='%'
    IF @Cod_ven=''
        SET @Cod_ven='%'
    IF @Estado=''
        SET @Estado='%'
    SELECT top 50 Num_fac,Fec_fac,Fec_can,Cod_cli,Raz_soc_cli,Ruc_cli,
    Departamento,Provincia ,Distrito, Cod_ven, Ape_ven ,Nom_ven ,Total, Estado
    from
    ( SELECT Num_fac,Fec_fac,Fec_can,Cod_cli,Raz_soc_cli,Ruc_cli,
    Departamento,Provincia ,Distrito, Cod_ven, Ape_ven ,Nom_ven ,Total, Estado ,
    ROW_NUMBER() OVER (ORDER BY Num_fac DESC) AS NumOrden from vw_VistaFacturas
    where
        Cod_cli like '%' + @Cod_cli + '%' and Cod_ven like '%' + @Cod_ven + '%'
        and Estado like '%' + @Estado + '%'
    ) AS CONSULTAPAGINADA
    WHERE NumOrden BETWEEN @PageSize * @Numpag + 1 AND @PageSize * (@Numpag + 1)
    ORDER BY Num_fac desc
END
```

/ LABORATORIO

Laboratorio 3: Implementar paginación local y paginación con procedimientos almacenados



En este laboratorio aprenderemos a implementar paginación local (con las características del control GridView) y la paginación con procedimientos almacenados.

/ CONCLUSIONES

- Las capas son reutilizables y nos permiten independizar la capa de presentación.
- Es importante interactuar con AJAX (u otra herramienta) que consolide una interface amigable para el mantenimiento de una tabla.
- Es importante paginar para poder mostrar de manera amigable y en secuencia de páginas el listado de los registros.

PÁGINAS WEB Y OTROS RECURSOS DE INTERNET

Web

- MICROSIERVOS: “Paginación: un recorrido por diversas soluciones adoptadas en el diseño de sitios web”
Recuperado de <https://www.microsiervos.com/archivo/internet/paginacion-soluciones-diseno-web.html>
Fecha de consulta: 23 de Mayo del 2023.
- HDELEON.NET: “Como hacer paginado con SQL Server | Transact SQL | Procedimiento almacenado”
Recuperado de <https://www.youtube.com/watch?v=q88GXsED5xE>
Fecha de consulta: 25 de Mayo del 2023.
- SQLSHACK: “Introducción a paginación”
Recuperado de : <https://www.sqlshack.com/es/sql-server-2012-introduccion-paginacion/>
Fecha de consulta: 25 de mayo del 2023.